

Reproducibility Log - Semi-Supervised Phylogenetic Inference

Felix Weik

Abstract. "Toward a Semi-Supervised Learning Approach to Phylogenetic Estimation" by Silvestro et al. introduces a semi-supervised deep learning (DL) model to predict heterogeneity rates given nucleotide alignments with a fixed number of taxa and sites. The paper provides conceptual thoughts plus an implementation of said model, called PhyloRNN. The authors show that using the predicted heterogeneity rates of PhyloRNN improves the posterior distribution and tree topology, plus speeds up the inference process. This log documents my attempt to reproduce the paper's results. I additionally assign site-wise heterogeneity rates using entropy measures in place of PhyloRNN, namely Shannon and Normalized Shannon Entropy. Executing the Ranking Test of the paper, I show that using Normalized Shannon consistently outperforms PhyloRNN.

1 Paper Overview

1.1 Summary

Silvestro et al. [5] introduce a DL model, called PhyloRNN, to infer heterogeneity rates from a multiple-sequence nucleotide alignment. The authors claim that their approach improves parameter validation and identification in comparison to probabilistic approaches such as Gamma [7] or Free Rates [6]. Probabilistic approaches rely on assumptions regarding underlying modeling processes. A DL model is able to accommodate arbitrary complexity by assigning site-wise rates as it is not limited to mathematical assumptions on the data. Thus, a DL model is able to achieve a better representation of the underlying biological process.

The results show that their model predicts heterogeneity rates more accurately. This leads to a better posterior distribution of the inference process, a better weighted RF-Distance, and sometimes a better general RF-Distance. Additionally, the authors claim their approach speeds up the inference process.

To prove these claims the authors conducted three tests:

General Rate Comparison. Assuming a true underlying heterogeneity model, for example Gamma, they calculate the mean squared error and R^2 score of the inferred rates when compared to the simulated true rates. The inferred rates stem either from a probabilistic inference approach or the PhyloRNN predictions. The results show an improved MSE and R^2 score using the fixed rates predicted by PhyloRNN.

Impacts on Tree Likelihood. Silvestro et al. conducted a ranking process comparing the likelihood of the true tree to 50 random samples drawn from the inferred posterior. These trees are ranked with decreasing likelihood. Following the logic of the paper, the higher the true tree ranks, the more often it is sampled from said distribution. Thus, the better the true tree ranks the better the inferred distribution. The authors show that the true tree ranks better using PhyloRNN predictions compared to rates from the Gamma Model.

Impacts on Tree Topology. The paper compares the true trees with inferred and true rates using both the unweighted and the weighted Robinson–Foulds distance (RF). There is no substantial improvement in the unweighted RF-Distance when using PhyloRNN rates compared to Gamma and Free Rates during inference. The weighted RF-Distance showed an improvement when using PhyloRNN. The authors conclude PhyloRNN helps in finding better branch lengths.

2 Existing Implementation

2.1 Sources

The implementation of Silvestro et al. is published on GitHub. Additional data including a single *.fasta* file [1], which is presumably the aligned clownfish genome, more scripts, and more detailed result tables are found on Dryad. The name of the clownfish alignment is `Chr1.WGAlign.FromBam.Filtered.fasta`.

2.2 Existing Code Base

The authors describe the codebase as two general modules: a simulator to generate realistic multiple sequence alignments (MSAs), and a DL module that executes the training process on those datasets to make predictions. In general, the implementation follows a predefined workflow. Outputs serve as inputs for the next step implemented in a different file. Many files are unlabeled, duplicated, or contain redundant code blocks.

Data Simulation. Silvestro et al. require 10000 datasets to train a single model. For this purpose they implement a simulation module generating a nucleotide MSA with a fixed number of taxa and sites.

In the implementation, the user can choose number of taxa and length of the alignments. The substitution model uses random base frequencies drawn from a symmetric Dirichlet distribution. A parameter determines whether substitution is random for each block or for the entire alignment. Per-site substitution rates are assigned according to one of five heterogeneity models. The nucleotide alignment is then produced by the external tool *Seq-Gen* [4]. The user can choose the degree of parallelization via the number of available CPU cores. Simulations output a NumPy archive containing features, per-site rates, and metadata.

Model Construction and Training. The network builds on TensorFlow/Keras. Its central component is a bidirectional recurrent network (LSTM), which allows information extraction from sequential data in both directions. Therefore, the model can extract information from neighborhood relations. Training uses both a definable validation split and number of epochs. This requires the NumPy archive from the previous step and the number of sites and taxa set as parameters. The model is saved in TensorFlow's *SavedModel* format.

Prediction and Rate Estimation. PhyloRNN generates predictions, which are then further analyzed using either *RevBayes* [3] or *PhyML* [2]. This paragraph focuses on the *RevBayes* workflow. Running predictions requires a trained model and an alignment to run predictions on. The alignment can either be a real-world example or generated with the simulation module described above. The alignment must have the same number of taxa and sites as the training data. Other dimensions require training a new model.

Predictions return the per-site rates stored in an array. This workflow generates three variations of *RevBayes* scripts:

- A standard Gamma model plus GTR.
- A script taking the predicted or calculated per-site rates using them directly. This infers all parameters but the heterogeneity rates. The implementation calls the predicted rates *DL*.
- Discretized site rates using n bins. The implementation's standard value is five (see 4.2). Each site is assigned to one of the n bins by picking the lowest distance bin given the predicted rate. As it therefore is a discretized version of *DL* Silvestro et al. call it *DL5d* in the implementation.

Those scripts use predefined text blocks that are activated, deactivated, or filled in by placeholders and function parameters. The resulting *RevBayes* scripts are executed manually in the next step.

RevBayes Evaluation. Executing each of the generated *RevBayes* scripts picks 10000 MCMC samples returning five text files.

- *.trees* contains 1000 post-burn-in trees stored as Newick-trees including the branch length, Likelihood, Prior, and Posterior.
- *.log* contains Posterior, Likelihood, Prior, and all parameters for every ten iterations.
- *MAP.tre* contains the single maximum a posteriori tree found in the *.trees* file.
- *MCC.tre* contains the maximum clade credibility tree, generated from the *.trees* file.
- *CT.tre* contains the consensus tree, again, generated from the *.trees* file.

Published example results are on GitHub.

PhyML Evaluation and Comparison. This paragraph focuses on the *PhyML* workflow. There is no published implementation for performing either the tests on the tree likelihood (1.1) or tree topology (1.1). This log focuses on the per-site rate part of both paper and implementation. `compare_rnn_phyml` compares per-site rates and tree lengths between the predictions, Free Rate, and Gamma models. The function calculates the mean squared error (MSE) and the R^2 score, thus corresponding to the general model comparison (1.1). It also generates plots comparing the predicted / calculated rates to the true rates. For more details see Section 3.3.

2.3 Setup

I use a Surface Laptop Studio with 16GB RAM, no graphics card, and an 11th Gen Intel Core i5-11300H, running Ubuntu 24.04.4 LTS.

The easiest setup method is using Anaconda / Mamba as it resolves package dependencies (see 3.1). Execute `mamba install -c conda-forge -c bioconda phylornn` to install all required package dependencies, as recommended by Silvestro et al. I additionally recommend creating a dedicated virtual environment for this purpose. The package versions are listed in `requirements.txt` on GitHub. The *RevBayes* workflow requires a running installation of *Seq-Gen* and *RevBayes*. *Seq-Gen* was installed using `apt`, *RevBayes* by taking release v1.4.1 from GitHub.

Using the built-in comparison method mentioned in 2.2 requires a running installation of *PhyML*, which is installable using `apt`. Following the paper’s implementation, I use *PhyML*.

3 Extending Implementation

3.1 Problems

The implementation revealed several problems, described below.

Different Codebases. Silvestro et al. published two different codebases containing different scripts. GitHub, as mentioned in 2.2, contains modules for simulation, training, prediction, and evaluation. This is the codebase the authors refer to in the paper. Dryad holds a `.zip` folder called `data_and_scripts` containing similar modules using different naming conventions. It additionally contains pretrained models for different sizes, more specifically 100 sites for 20 taxa, and 1000 sites for 28 and 50 taxa. As the training data for those models is not further specified, I assume it was trained with data from the simulation modules.

I combined both codebases taking the pretrained models from Dryad while using utils, simulation modules, and evaluation modules from GitHub.

Conventions. The issue above is just one of the symptoms summarized here as convention violations.

- Incomplete error handling. This includes a Keras fallback when calling models from faulty directories leading to wrong error messages, division-by-zero exceptions, wrong array accesses, and usage of deprecated functions. These fallbacks only appear for false parameters. For correct function parameters and directories those issues do not appear.
- Hard-coded directories without documentation. This made it unclear to find out what they were used for.
- Dead files containing unused functions and hard-coded directories. Those files probably stem from the construction process of the initial codebase and only meant readability issues.
- Deprecated package versions for newer Python versions. The authors use tilde version specifier, that allow for package releases within the same minor version. The required minor version for TensorFlow of 2.9.x, for example, does not exist for Python 3.9+. Use Anaconda / Mamba to resolve those dependencies. Silvestro et al. summarize their requirements in `requirements.txt`.
- Near-duplicate files differing only in a few parameters make the intended workflow unclear. Often, two versions of the same file exist where one script has large blocks of commented code which the other script executes. In the simulation module, for example, it is unclear which datasets train and validate the model, and how they act on each other. As I rely on the pretrained models, there was no further investigation done on this particular topic.

Misleading Workflow. The published implementation contains unused functions and scripts. Some misleadingly named files, however, have led to major confusion in my development process.

`run_predictions`, for example, trains a model from scratch and performs predictions on the training data. They then use the same dataset from training to test the model, which is a cardinal sin in machine learning. Many such misleading files exist in the codebase.

Missing Data and Validation. Dryad contains a single `.fasta` file. It is presumably the clownfish genome mentioned in the paper of Silvestro et al. [5]. `data_and_scripts` contains a test workflow analyzing this genome using the pretrained model for 28 taxa and 1000 sites.

Furthermore, there is no predefined dataset for model training or defined set given the pretrained models. The given implementation uses seed 1234 for training data simulation, and seed 4321 for unseen prediction MSAs. Yet, there is no validation whether this returns the dataset used in the paper. Therefore, I cannot verify whether this is the same dataset used in the paper.

Supplementary Figure 11 mentioned in the paper is missing. Given the context information from the paper it should be an evaluation figure showing the results of the ranking test.

3.2 Introducing Entropy

We wanted an easier way to assign site-wise heterogeneity rates without introducing the bottleneck of machine learning. I extended the existing implementation with a benchmark against entropy measures. Parts of my implementation were generated with help of Claude Opus 5.5 and DeepSeek V4 Flash. The implementation uses the Shannon Entropy (SH) and the derived Normalized Shannon Entropy (NSH). It counts the different bases from the simulation stored as a NumPy array assigning site-wise values. Both measures are then used as heterogeneity rates.

3.3 Updated Workflow

The extended implementation used most of the existing codebase (see 2) adding test and comparison modules. All steps are similar to the higher-level explanation of the existing code base (see 2.2). This leads to the following workflow in my implementation:

1. Training Data Simulation. Handled in file *1a.simulate_train_data_revb*, generates an arbitrarily large training dataset with arbitrary number of sites and taxa. These values are set with the parameters `n_taxa` and `n_sites` of the `phylornn.simulator` functions. All other parameters remain unchanged from the implementation by the authors. File *1.simulate_train_data* additionally generates validation data for model training. I use pretrained models allowing me to skip this and the following step.
2. Model Training. This step is similar to the paragraph on model construction and training. *2.train_model* trains and saves a model given the training set from the simulation step. As I was not able to train any models, all parameters in the published file are the same to the version of Silvestro et al.
3. Prediction and Rate Estimation. *3.run_predictions_revb* runs predictions on a new MSA assigning site-wise heterogeneity rates. These rates are stored in an array and passed to a RevBayes script. Execute the scripts manually or via another script to receive the five files mentioned above.
4. PhyML Evaluation. The comparison modules estimating maximum likelihood against the prediction rates (see 2.2) are not called within the given workflow of the codebase and thus have to be integrated into the new workflow. *4a.run_phyml_comparison* integrates this function. It takes `n_sims` as a parameter specifying the number of simulations per run. Each simulation will result in a single data point in the plot. I additionally changed `n_taxa` and `n_sites` of the `phylornn.simulator` function to match the model's requirements. Then, for function `compare_rnn_phyml`, I changed the `model_file` parameter to match the model name, `output_dir` to match an existing directory I want to save my results in, and `n_taxa` to match the number of taxa for the used model and the simulated alignment. All other parameters remain unchanged.

5. RevBayes Evaluation. *complete_workflow_revb* summarizes the RevBayes workflow running the complete pipeline from start to end. Given a pretrained model, it simulates a new data set of arbitrary size, predicts rates and executes the generated RevBayes scripts as in step 3, and performs the ranking test from step 4 afterwards. The ranking test recalculates the likelihood of the true tree given the calculated heterogeneity rates. This likelihood is then ranked given the likelihoods of the RevBayes *.log* file. Following the logic of Silvestro et al., the higher the true tree ranks, the better the model. For this I altered `n_sites` and `n_taxa` in `phylornn.simulator` to match the model requirements. Then, I set `start_sim` to 0, which is used for naming the result files later on. I also changed `n_sim` to 1, as I want only one simulation per run. Increasing this number will return multiple output files per model, indexed by the run number.

The extended implementation is published on GitHub.

4 Results

This Section is structured into two distinct parts: First, I executed the given implementation of the *PhyML* workflow. This produces plots comparing the underlying heterogeneity model of the simulation to the applied heterogeneity model for inference. The inference models are Gamma [7], Free Rates [6], and PhyloRNN [5]. The following Subsection 4.1 focuses on comparing the inferred values to the results of Silvestro et al. [5]. This is equivalent to Section *Performance of the PhyloRNN Model* from the paper, described in 1.1. The plots comparing the tree length do not consider the RF-Distance. As the plots still show some trends, they are briefly discussed.

Second, Subsection 4.2 focuses on the comparison between the added entropy measures compared to both PhyloRNN and true rates. I implemented the ranking test as described in 3.3 or the *Impact on Tree Likelihood* section from the paper.

4.1 Result Reproducibility

Performance of the PhyloRNN Model I have linked the raw simulation results and example logs over 10 runs on my GitHub. Each run contains 100 simulations on 28 taxa with 1000 sites, executed on starting seeds 4321 and 4320. A single run finishes in approximately 20 minutes on my machine. `results/phyml/vanilla/*` contains the complete results of each run.

Impact of Changing Seeds. I wanted to check whether changing the seed for generating the alignment to run predictions on changes the predictions. For this, I tested 8 simulations, namely simulation 0 to 7, on seed 4312. The remaining two simulations use seed 4320. Using either the same seed or a different seed led to different predictions and results. As mentioned in 3.1, I can not verify whether this returns the same data used in the paper.

Model Results. Table 1 summarizes the per-site rate predictions using the same metrics as the paper, namely R^2 and MSE. PhyloRNN performs better in 6 out of the 11 tested rate models, Free Rate wins 4-times, and Gamma 1-time. PhyloRNN, however, performs worse than Free Rates in the overall comparison. This contradicts the paper where PhyloRNN performed the best overall.

Rating the model performance per rate model gives a similar outcome to the paper’s result. Most models achieve the same relative performance order as the paper. Two rate models, namely GBM uncorrelated and Spike-and-Slab uncorrelated, have a different winner in the paper. For some models, especially Codon uncorrelated, the ranking is the same, yet the absolute values are far worse in the reproduction. In the paper, PhyloRNN scores an R^2 of 0.942 for Codon uncorrelated, while my reproduction only yields an R^2 of 0.585.

Training a model from scratch could change the results drastically. The predictions in this test stem from the *t28_s1000* model initially trained for the clownfish genome. Also, increasing the number of test runs might yield results closer to the paper.

Impacts on Tree Topology I compare the Mean absolute percentage error (MAPE) and R^2 values regarding the models over all 10 runs.

The general trend opposes the stated observation of PhyloRNN improving the tree topology. My reproduced results show that Free Rates performs better. Its mean tree-length MAPE of 0.115 is better than Gamma’s with 0.129 and far better than PhyloRNN with 4.95. PhyloRNN performs far worse with large absolute error and high variance. It is competitive for Gamma autocorrelated and the mixed model.

Figure 1 plots the estimated tree length (on the y-axis) to the true tree length on the x-axis. Each point represents a single comparison. A perfect estimation, meaning estimated length being equal to the true length lies on the main diagonal.

This visualizes the worse performance of PhyloRNN on tree lengths as it underestimates the true tree length in more cases than its competitors. Those tests use the *t28_s1000* model initially trained by Silvestro et al. for predicting rates on the clownfish data. I could not verify whether this issue of underestimating tree lengths stems from the probably overfitted model for clownfish data without generalization on unseen data.

4.2 Entropy Results & Comparison

Naive Implementation The implementation uses Shannon (SH) and Normalized Shannon Entropy (NSH) as site-wise heterogeneity rates. It also considers the Gamma Model (G), the predicted rates from the model (DL), plus a discretized version considering only five heterogeneity classes with each site being assigned the closest class (DL5d). The evaluation of this implementation resembles the ranking test from the paper.

I ran 10 tests on 20 taxa and 100 sites which finished in approximately 30 minutes each. Each RevBayes run picks 10000 samples per model. The output

Rate Model	Method	MSE (mean $\pm$ std)	R ² (mean $\pm$ std)
Bimodal_autocorr.	Gamma	0.380 $\pm$ 0.084	0.098 $\pm$ 0.068
	FreeRate	0.321 $\pm$ 0.059	0.081 $\pm$ 0.050
	RNN	0.194 $\pm$ 0.048	0.157 $\pm$ 0.078
Bimodal_uncorr.	Gamma	0.430 $\pm$ 0.129	0.076 $\pm$ 0.042
	FreeRate	0.354 $\pm$ 0.082	0.069 $\pm$ 0.038
	RNN	0.415 $\pm$ 0.079	0.017 $\pm$ 0.005
Codon_autocorr.	Gamma	0.514 $\pm$ 0.086	0.236 $\pm$ 0.131
	FreeRate	0.574 $\pm$ 0.127	0.227 $\pm$ 0.127
	RNN	0.480 $\pm$ 0.075	0.323 $\pm$ 0.146
Codon_uncorr.	Gamma	0.505 $\pm$ 0.051	0.236 $\pm$ 0.121
	FreeRate	0.560 $\pm$ 0.110	0.225 $\pm$ 0.121
	RNN	0.369 $\pm$ 0.083	0.585 $\pm$ 0.213
GBM_autocorr.	Gamma	0.176 $\pm$ 0.051	0.035 $\pm$ 0.021
	FreeRate	0.252 $\pm$ 0.183	0.033 $\pm$ 0.026
	RNN	0.106 $\pm$ 0.027	0.312 $\pm$ 0.123
GBM_uncorr.	Gamma	1.653 $\pm$ 0.476	0.254 $\pm$ 0.075
	FreeRate	1.234 $\pm$ 0.277	0.284 $\pm$ 0.100
	RNN	1.800 $\pm$ 0.520	0.068 $\pm$ 0.025
Gamma_autocorr.	Gamma	1.832 $\pm$ 0.198	0.345 $\pm$ 0.114
	FreeRate	1.467 $\pm$ 0.339	0.381 $\pm$ 0.138
	RNN	1.211 $\pm$ 0.208	0.572 $\pm$ 0.052
Gamma_uncorr.	Gamma	2.389 $\pm$ 0.623	0.338 $\pm$ 0.050
	FreeRate	2.125 $\pm$ 0.595	0.376 $\pm$ 0.083
	RNN	2.651 $\pm$ 0.621	0.116 $\pm$ 0.030
Mixed-model	Gamma	1.589 $\pm$ 0.672	0.000 $\pm$ 0.001
	FreeRate	1.382 $\pm$ 0.515	0.000 $\pm$ 0.001
	RNN	1.441 $\pm$ 0.519	0.000 $\pm$ 0.001
Spike-and-slab autocorr.	Gamma	0.441 $\pm$ 0.116	0.058 $\pm$ 0.031
	FreeRate	0.356 $\pm$ 0.087	0.063 $\pm$ 0.041
	RNN	0.349 $\pm$ 0.113	0.103 $\pm$ 0.088
Spike-and-slab uncorr.	Gamma	0.597 $\pm$ 0.100	0.091 $\pm$ 0.029
	FreeRate	0.632 $\pm$ 0.217	0.157 $\pm$ 0.095
	RNN	0.663 $\pm$ 0.103	0.011 $\pm$ 0.008
Overall	Gamma	0.893 $\pm$ 0.027	0.124 $\pm$ 0.008
	FreeRate	0.791 $\pm$ 0.063	0.132 $\pm$ 0.019
	RNN	0.823 $\pm$ 0.029	0.104 $\pm$ 0.013

Table 1. Per-Site Rate Prediction Summary: MSE and R² across 10 runs. Best performing model marked in bold for each rate model.

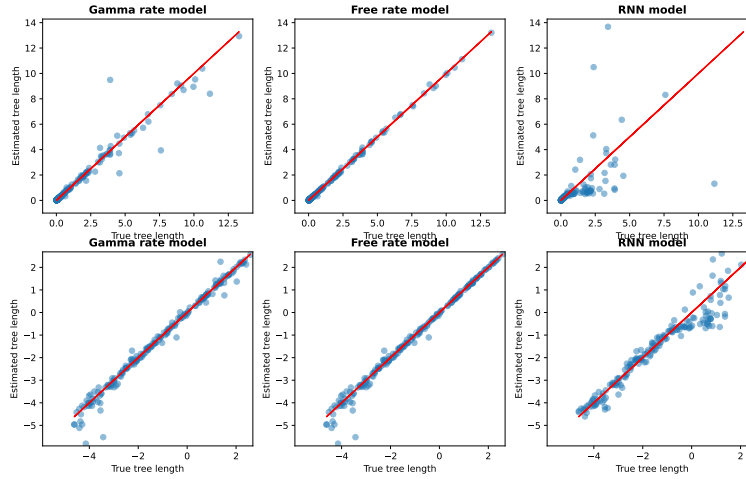


Fig. 1. Scatter plot comparing estimated against true tree length. Upper row plots the raw data, lower row same data but in log scale.

logs consider every 10 iterations. Despite the obvious drawbacks of this procedure, it is to stay aligned with the paper’s methods. Note that AIC and BIC score comparisons are left out, as the models, namely DL, SH, and NSH, assign site-wise parameters and therefore the same number of parameters. It would be a likelihood comparison through the backdoor.

Runtime. Table 2 shows both mean and median RevBayes runtime for each model through all 10 runs.

Contrary to the paper’s claim, using the predicted rates from PhyloRNN did not speed up tree inference. All methods assigning site-wise rates (SH, NSH, DL) are approximately 11-14 times slower than Gamma and DL5d. This may be due to RevBayes issues handling large rate arrays, or general hardware limitations. Still, it shows a general trend of methods approximating the rates instead of assigning site-wise rates achieving better runtimes.

True Tree Ranking. 10000 post burn-in samples logged every 10 iterations lead to 1000 samples. Sorting the log-likelihoods of those trees, the implementation returns the index of the true tree given the predicted / calculated rates in the sorted array. Therefore, tree 0 has the best log-likelihood and tree 1000 the worst. Table 3 shows the results for both the mean and the median index for each model.

Note the gap in median and mean index for SH. It stems from run 5, ranking the SH tree at index 245. As the other runs return worse indices, this outlier skews the mean.

Model	Mean	Median
G	36.0	37.0
DL	404.9	348.0
DL5d	28.4	27.5
SH	406.9	363.0
NSH	398.0	358.5

Table 2. Mean and median RevBayes MCMC runtimes across all 10 datasets, in seconds. Additional time required for predicting / calculating the rates and training the model were not considered.

The paper’s implementation assigns the reference / true tree a Gamma model. Therefore, the true tree using the Gamma rates ranks the best on average. The recalculated tree was ranked lowest for SH. This means, the true tree is among the worst in the posterior given the calculated SH rates. Both PhyloRNN flavors (DL, DL5d) lead to the true tree ranking in the lower 30% performing better than SH. NSH ranks the true tree best among the site-wise models.

Following the arguments from the paper, the higher the true tree ranks the better the underlying inferred distribution. This means using NSH instead of DL, DL5d, or SH results in a better tree distribution.

Model	Mean	Median
G	278.7	131.0
DL	700.3	776.0
DL5d	705.5	789.5
SH	858.4	995.0
NSH	601.9	612.0

Table 3. True tree ranking in the posterior likelihood distribution ($N = 1001 = 1000$ posterior samples + 1 true tree). The better the likelihood, the better it ranks in the sorted posterior log-likelihood array (0 = best/first, 1000 = worst/last).

Log-Likelihood Difference of the True Tree. The paper’s implementation uses a Gamma model as reference for the true tree. Therefore, there will not be any change in log-likelihood for the Gamma model, which means G is left out in the result Table 4 below.

Both PhyloRNN methods (DL, DL5d) show an on average deteriorating log-likelihood for the true tree using the predicted rates. For SH, there is a minor increase in its log-likelihood while NSH shows large positive differences. Also, NSH improves the log-likelihood for every run. This is not the case for the other models. This means the true tree using NSH rates fits the data better.

Run	Δ LL (DL)	Δ LL (DL5d)	Δ LL (SH)	Δ LL (NSH)
0	+7.0568	+4.5450	+12.9853	+29.4872
1	-4.8822	-6.7703	-8.9271	+39.1874
2	-18.3414	-19.5198	-13.8992	+22.0911
3	+12.7823	+10.8968	+28.4942	+38.8171
4	+9.8197	+5.7683	-22.5586	+44.1731
5	-7.0277	-10.7287	+14.8148	+15.4944
6	-2.1707	-2.5948	-0.9410	+45.2567
7	+3.7518	-0.9129	-12.9080	+11.3311
8	-17.2802	-17.9805	-3.2134	+2.8545
9	-0.7517	-3.2998	+23.3689	+23.8606
Mean	-1.7043	-4.0597	+1.7216	+27.2553
Median	-1.4612	-2.9473	-2.0772	+26.6739
Min	-18.3414	-19.5198	-22.5586	+2.8545
Max	+12.7823	+10.8968	+28.4942	+45.2567

Table 4. Difference of the true tree log likelihood change evaluated under the according model. Positive means the true tree has higher log-likelihood given the predicted / calculated rates. Note the Gamma tree being the reference meaning no improvement in recalculation. Thus, G is left out here.

5 Conclusion

Both published implementations of Silvestro et al. do not consider all tests from the paper. Given the three tests conducted in the paper (see 1.1), only the general rate comparison is implemented in the published codebase. Yet, the codebase provides the generating scripts but not the generated data the authors based their training and evaluation on. Also, I could not verify whether the data used by my extended implementation is the same as in the paper’s results.

The reproduced results are not equivalent to the results from the paper: Free Rates performed better in the overall performance comparison (see 1.1, 4.1), and the predicted tree lengths from PhyloRNN were far worse than shown in the paper. Also, Gamma outperformed PhyloRNN in the ranking test, contrary to the paper’s results (see 1.1, 4.2). This may be due to Gamma being used as the reference model, which is, however, the same setup as in the paper.

Further, I introduced a different approach to assign site-wise heterogeneity rates using Shannon Entropy and the Normalized Shannon Entropy. RevBayes runtime stayed the same when using PhyloRNN or SH / NSH rates, yet the log-likelihood of the true tree improved for NSH. Also, implementing the ranking test from the paper, using Normalized Shannon rates led to better ranking results for the true tree than using predicted rates from PhyloRNN. This also is different to the paper, where PhyloRNN is claimed to speed up the inference process. Considering the predicted tree lengths, PhyloRNN performed far worse than Gamma and Free Rates.

Therefore, I could not reproduce the paper’s results given the published code and the pretrained models.

Silvestro et al. introduce an LSTM called PhyloRNN. Each instantiation is trained on 10000 datasets and is fixed on the number of sites and taxa of the MSA. LSTMs are known for being especially black-boxy. Those points restrict both usability of the model and interpretability of the results. Contrary to this, NSH uses a clear formula, does not require predefined size limits, is applicable to non-nucleotide alignments as well, and does not require training meaning optimization of a loss function. This not only improves usability and interpretability, but was also shown to outperform PhyloRNN in the ranking test (see 4.2). Therefore, one should rely on Gamma or Normalized Shannon.

6 Future Research

More tests should be executed including larger datasets. One should try to reproduce the results on the clownfish genome published on Dryad. One should train a model from scratch testing how different training data affects its prediction results. Also, retesting the impacts on tree topology from 4.1 is necessary to validate the current values.

Setup. Silvestro et al. assign Gamma as the reference model for the RevBayes workflow. Therefore, Gamma ranking best (see 4.2) might be due to the setup, not because of the other models performing worse. One should test how the models behave for different reference models.

New Measures. Shannon Entropy is not centered around 1 and assigns 0 to conserved sites. Simple workarounds include normalization, as shown by using NSH, or adding a small constant to 0 rates. One could replace them with measures that have these properties built in by construction. Hill Numbers, especially the exponential of the Shannon Entropy, have these properties by construction.

Neighborhood Relations. PhyloRNN is a recurrent neural network, more specifically a bidirectional LSTM. Thus, it is able to capture neighborhood relations in both directions given sequential data, for example MSAs. One could alter the introduced entropies to also capture neighborhood relations.

A running average on the calculated entropy rates is similar to the momentum used in gradient descent. By choosing appropriate weights, one tackles low assigned values in more conserved regions and receives a simple model of neighborhood relations. Also, it is cheap to compute. It is, however, limited to a single direction (backwards) and cannot capture relations in front of current site. Also, it decreases interpretability of the results.

Alternatively, one could use a sliding window as it captures both directions. Of course, this could be improved and fitted on the alignment as well, for example by introducing edge handling or fitting sliding windows to codons only capturing explicit dynamics. This, again, increases complexity and introduces new steps taken before performing the inference.

7 Acknowledgments

Auch wenn eine Danksagung für eine Seminararbeit etwas viel ist, möchte ich mich nochmal herzlich bei Dimitri und Alexis für die tolle, aufopferungsvolle Betreuung bedanken. Dir, Dimitri, nochmal vielen Dank für deine Zeit, deine Geduld und dein fantastisches Feedback auch zu ungnädigen Uhrzeiten.

References

1. Baxevanis, A.D., Ouellette, B.F.F. (eds.): *Bioinformatics: A Practical Guide to the Analysis of Genes and Proteins, Methods of Biochemical Analysis*, vol. 39. John Wiley & Sons, New York (1998)
2. Guindon, S., Dufayard, J.F., Lefort, V., Anisimova, M., Hordijk, W., Gascuel, O.: New algorithms and methods to estimate maximum-likelihood phylogenies: assessing the performance of phylml 3.0. *Systematic biology* **59**(3), 307–321 (2010)
3. Höhna, S., Landis, M.J., Heath, T.A., Boussau, B., Lartillot, N., Moore, B.R., Huelsenbeck, J.P., Ronquist, F.: Revbayes: Bayesian phylogenetic inference using graphical models and an interactive model-specification language. *Systematic biology* **65**(4), 726–736 (2016)
4. Rambaut, A., Grass, N.C.: Seq-gen: an application for the monte carlo simulation of dna sequence evolution along phylogenetic trees. *Bioinformatics* **13**(3), 235–238 (1997)
5. Silvestro, D., Latrille, T., Salamin, N.: Toward a semi-supervised learning approach to phylogenetic estimation. *Systematic Biology* **73**(5), 789–806 (2024)
6. Soubrier, J., Steel, M., Lee, M.S., Der Sarkissian, C., Guindon, S., Ho, S.Y., Cooper, A.: The influence of rate heterogeneity among sites on the time dependence of molecular rates. *Molecular Biology and Evolution* **29**(11), 3345–3358 (11 2012). <https://doi.org/10.1093/molbev/mss140>, <https://doi.org/10.1093/molbev/mss140>
7. Stamatakis, A.: Phylogenetic models of rate heterogeneity: A high performance computing perspective. proceedings of the 20th ieee/acm international parallel and distributed processing symposium (ipdps 2006), 2006 high performance computational biology workshop, proceedings on cd and available online; april 2006; rhodos, greece. Rhodos, Greece (2006)